

# Universal Real-Time Infrastructure Monitoring and Observability Platform

*A Summer Internship Project Report*

Submitted by

**Dhruv Rathore**

Roll Number: 23052240

B.Tech Computer Science and Engineering  
KIIT University, Bhubaneswar

Submitted in partial fulfilment of the Summer Internship Training at

**ISTRAC, Lucknow, ISRO**

**Project Guide:** Shri Anant Madhukar

**Designation:** Scientist / Engineer, ISRO

**Duration:** 11 May 2026 to 18 June 2026

**ISRO TELEMETRY, TRACKING AND COMMAND NETWORK (ISTRAC)**  
LUCKNOW, INDIA

## **Certificate of Completion**

This is to certify that **Dhruv Rathore**, Roll Number **23052240**, student of B.Tech Computer Science and Engineering, KIIT University, Bhubaneswar, has successfully completed the Summer Internship Training at **ISTRAC, Lucknow, ISRO**, during the period **11 May 2026 to 18 June 2026**.

During the internship, the trainee worked on the project titled **“Universal Real-Time Infrastructure Monitoring and Observability Platform”** under the direct supervision and guidance of **Shri Anant Madhukar**. The work involved the design and implementation of a distributed full-stack monitoring system for infrastructure metrics, alerting pipelines, real-time dashboarding, and host management.

The work submitted in this report is based on the project activities carried out during the internship period.

**Signature of Guide:** \_\_\_\_\_ **Date:** \_\_\_\_\_

Name: Shri Anant Madhukar

Designation: Scientist / Engineer, ISRO

Organization: ISTRAC, Lucknow, ISRO

# Acknowledgement

I express my sincere gratitude to **Shri Anant Madhukar** for his invaluable guidance, technical direction, and constant support throughout my summer training at ISTRAC, Lucknow. His expert inputs helped me understand how monitoring systems must be designed for professional technical environments where reliability, high observability, and operational clarity are critically important.

I am highly thankful to **ISTRAC, Lucknow, ISRO** for providing me the opportunity to work on a project connected to real infrastructure monitoring concerns. The internship helped me understand the role of Tracking, Telemetry, and Command (TTC) operations, and the supporting ground infrastructure that enables mission-related work. The project gave me practical exposure to distributed systems, high-throughput data pipelines, native monitoring agents, databases, and dashboard design.

I also extend my thanks to **KIIT University, Bhubaneswar**, and the Department of Computer Science and Engineering for encouraging industry and research institution exposure through internship training. The academic foundation provided by the university helped me approach the project with the required knowledge of software engineering, databases, operating systems, networks, and web technologies.

# Executive Summary

The project titled **“Universal Real-Time Infrastructure Monitoring and Observability Platform”** was developed during the Summer Internship at ISTRAC, Lucknow, ISRO. The objective was to design and implement a full-stack infrastructure monitoring platform functionally similar to Zabbix. The system monitors servers, virtual machines, containers, network services, and host-level metrics in real time through lightweight monitoring agents.

The work was carried out in two distinct phases. During the first three weeks, a universal real-time event ingestion platform was developed. It supported HTTP and Kafka ingestion, dynamic parsing of JSON, CSV, XML, Syslog, and Key-Value formats, PostgreSQL persistence, JWT authentication, API keys, and WebSocket-based live dashboard updates. This phase established the distributed system foundation required for the final monitoring platform.

During the final three weeks, the architecture was redesigned for infrastructure monitoring. A zero-dependency Node.js monitoring agent was created to collect CPU, RAM, disk, network, process, uptime, and service metrics on Windows and Linux. Agent check-ins are received by a Fastify-based Agent API and published to Kafka. A Metrics Processor consumes the Kafka stream, writes metric values to PostgreSQL and ClickHouse, evaluates alert rules, opens or resolves problems, updates host status, and publishes live events through Redis. A WebSocket Gateway then delivers updates to the Next.js dashboard.

The final platform includes host inventory, host detail pages, latest metric data, alert rules, active problems, alert history, maintenance windows, agent management, availability reports, system reports, user management, audit logs, and profile management. Validation confirmed that agent metrics reached the dashboard in under 2 seconds, alert rules fired correctly, problems auto-resolved when thresholds cleared, and multiple hosts could be monitored simultaneously.

# Abstract

This report presents the design and implementation of a real-time infrastructure monitoring and observability platform developed during summer training at ISTRAC, Lucknow, ISRO. The platform collects host metrics using lightweight monitoring agents, processes metric payloads through a Kafka-based pipeline, stores operational data in PostgreSQL, stores time-series data in ClickHouse, evaluates threshold-based alert rules, and displays live infrastructure status through a Next.js dashboard. The system was developed in two phases: first as a universal event ingestion platform and later as a Zabbix-style monitoring platform. The final prototype validates real-time metric collection, alert generation, auto-resolution, WebSocket dashboard updates, and multi-host monitoring.

**Keywords:** Infrastructure Monitoring, Observability, Kafka, PostgreSQL, ClickHouse, WebSocket, Zabbix, Node.js, Fastify, Next.js

# Contents

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                   | <b>7</b>  |
| 1.1      | Organization Overview . . . . .       | 7         |
| 1.2      | Project Background . . . . .          | 7         |
| 1.3      | Problem Statement . . . . .           | 7         |
| 1.4      | Need for the Project . . . . .        | 8         |
| 1.5      | Objectives . . . . .                  | 8         |
| 1.6      | Scope . . . . .                       | 8         |
| <b>2</b> | <b>Requirement Analysis</b>           | <b>9</b>  |
| 2.1      | Functional Requirements . . . . .     | 9         |
| 2.2      | Non-Functional Requirements . . . . . | 9         |
| 2.3      | Constraints . . . . .                 | 9         |
| 2.4      | Assumptions . . . . .                 | 10        |
| <b>3</b> | <b>System Analysis</b>                | <b>11</b> |
| 3.1      | Existing Systems Review . . . . .     | 11        |
| 3.2      | Phase 1 Analysis . . . . .            | 11        |
| 3.3      | Gap Analysis . . . . .                | 11        |
| 3.4      | Proposed Solution . . . . .           | 11        |
| <b>4</b> | <b>System Design</b>                  | <b>12</b> |
| 4.1      | High-Level Architecture . . . . .     | 12        |
| 4.2      | Microservices Architecture . . . . .  | 12        |
| 4.3      | Data Flow Design . . . . .            | 12        |
| 4.4      | Database Design . . . . .             | 13        |
| 4.4.1    | PostgreSQL Schema . . . . .           | 13        |
| 4.4.2    | ClickHouse Schema . . . . .           | 13        |
| 4.5      | Authentication Architecture . . . . . | 13        |
| 4.6      | Alert Engine Design . . . . .         | 13        |
| 4.7      | Monitoring Agent Design . . . . .     | 13        |
| <b>5</b> | <b>Implementation</b>                 | <b>15</b> |
| 5.1      | Technology Stack . . . . .            | 15        |
| 5.2      | Development Environment . . . . .     | 15        |
| 5.3      | Phase 1 Implementation . . . . .      | 15        |
| 5.4      | Phase 2 Implementation . . . . .      | 15        |
| <b>6</b> | <b>Testing and Validation</b>         | <b>17</b> |
| 6.1      | Test Strategy . . . . .               | 17        |
| 6.2      | Unit-Level Validation . . . . .       | 17        |
| 6.3      | Integration Testing . . . . .         | 17        |
| 6.4      | Performance Observations . . . . .    | 18        |

|           |                                     |           |
|-----------|-------------------------------------|-----------|
| 6.5       | Alert Engine Validation . . . . .   | 18        |
| <b>7</b>  | <b>Results and Discussion</b>       | <b>19</b> |
| 7.1       | Features Delivered . . . . .        | 19        |
| 7.2       | Performance Results . . . . .       | 19        |
| 7.3       | Comparison with Zabbix . . . . .    | 19        |
| 7.4       | Benefits Achieved . . . . .         | 19        |
| <b>8</b>  | <b>Challenges and Learnings</b>     | <b>20</b> |
| 8.1       | Technical Challenges . . . . .      | 20        |
| 8.2       | Architectural Evolution . . . . .   | 20        |
| 8.3       | Engineering Trade-offs . . . . .    | 20        |
| 8.4       | Lessons Learned . . . . .           | 20        |
| <b>9</b>  | <b>Future Enhancements</b>          | <b>21</b> |
| 9.1       | Immediate Next Steps . . . . .      | 21        |
| 9.2       | Medium-Term Roadmap . . . . .       | 21        |
| 9.3       | Scalability Opportunities . . . . . | 21        |
| 9.4       | Research Opportunities . . . . .    | 21        |
| <b>10</b> | <b>Conclusion</b>                   | <b>22</b> |
| <b>A</b>  | <b>Installation Guide</b>           | <b>23</b> |
| <b>B</b>  | <b>Deployment Guide</b>             | <b>24</b> |
| <b>C</b>  | <b>User Manual</b>                  | <b>25</b> |
| <b>D</b>  | <b>Agent Installation Guide</b>     | <b>26</b> |
| <b>E</b>  | <b>API Reference</b>                | <b>27</b> |
| <b>F</b>  | <b>Configuration Reference</b>      | <b>28</b> |

# Chapter 1

## Introduction

### 1.1 Organization Overview

ISTRAC, the ISRO Telemetry, Tracking and Command Network, supports mission operations by providing tracking, telemetry, and command services. TTC operations depend on a network of ground infrastructure components, communication systems, servers, software services, and operational tools. These components must remain highly available (99.999% uptime target) and observable because technical faults can affect larger mission support workflows.

Infrastructure monitoring is therefore an important part of technical operations. Monitoring systems help engineers detect abnormal CPU usage, memory exhaustion, disk pressure, network issues, stopped services, and delayed host responses. In such environments, the value of monitoring lies not only in collecting data but also in presenting actionable information in real time.

### 1.2 Project Background

The internship project was initiated as a distributed systems project involving real-time data ingestion, processing, persistence, and dashboarding. The first phase focused on general event ingestion. The second phase refined the direction toward infrastructure monitoring, where the system could collect structured host metrics and evaluate alert rules.

The final project scope was shaped around building a monitoring platform functionally similar to Zabbix. The goal was not to reproduce all features of a mature monitoring product but to implement core concepts such as hosts, agents, metrics, alert rules, problems, acknowledgements, historical charts, and live status updates.

### 1.3 Problem Statement

Technical environments often contain heterogeneous infrastructure. A single operational setup may include Linux servers, Windows systems, databases, containers, virtual machines, routers, and application services. When each system is monitored through separate commands or vendor-specific tools, engineers do not get a unified view of health and availability.

Manual checking cannot scale as the number of hosts increases. Delayed detection of resource exhaustion or service failure can increase operational risk. A unified monitoring platform is required to collect metrics continuously, evaluate thresholds automatically, and present real-time status to operators.



## 1.4 Need for the Project

A unified infrastructure monitoring system provides early fault detection, reduces manual checks, and improves operational awareness. It enables engineers to identify abnormal conditions such as high CPU load, memory pressure, disk usage, network traffic spikes, and service failures from a single dashboard.

The project was useful as a training exercise because it required the integration of operating system concepts, networking, distributed messaging, databases, authentication, WebSocket delivery, and frontend dashboard design. It also reflected the type of system engineering challenges that appear in technical institutions.

## 1.5 Objectives

- To design a distributed monitoring architecture based on microservices.
- To implement a lightweight monitoring agent for Windows and Linux.
- To collect CPU, RAM, disk, network, uptime, process, and service metrics.
- To process metric payloads through Kafka and persist them reliably.
- To implement threshold-based alert rules and problem management.
- To build a professional web dashboard with live updates and historical charts.

## 1.6 Scope

The project covers agent-based infrastructure monitoring, REST APIs, authentication, metric processing, time-series storage, alert evaluation, WebSocket updates, host management, problem management, and dashboard reporting.

The project does not include SNMP-based agentless monitoring, Kubernetes deployment, production-scale load testing, email or SMS notification delivery, SSL certificate monitoring, or network auto-discovery. These items are identified as future enhancements.

## Chapter 2

# Requirement Analysis

### 2.1 Functional Requirements

The functional specifications define the operational capabilities that the platform must support.

Table 2.1: Functional Requirements

| FR-ID | Requirement         | Description                                                             |
|-------|---------------------|-------------------------------------------------------------------------|
| FR-01 | Agent registration  | Register a monitored host and issue a secure agent key                  |
| FR-02 | Heartbeat ingestion | Receive host heartbeat signals every 10 seconds                         |
| FR-03 | Metric ingestion    | Receive full system metric check-in payloads every 30 seconds           |
| FR-04 | Host inventory      | Provide operations to list, create, update, and delete hosts            |
| FR-05 | Metric storage      | Write raw values concurrently to PostgreSQL and Click-House             |
| FR-06 | Alert rules         | Create warning and critical threshold rules for metrics                 |
| FR-07 | Problem workflow    | Open, acknowledge, and resolve problems automatically or manually       |
| FR-08 | Live dashboard      | Push metric updates to browser sessions using Web-Sockets ( $\leq 2$ s) |
| FR-09 | Historical charts   | Display historical trend charts for selected ranges using Recharts      |
| FR-10 | User management     | Provide administrator controls, session management, and API keys        |

### 2.2 Non-Functional Requirements

The technical and quality expectations of the system are outlined below.

### 2.3 Constraints

Development was completed over a six-week duration. The workspace was limited to local system capabilities running Windows 11, PowerShell, Node.js 20, and Docker Desktop. Relational schemas utilize a seeded single-tenant configuration to fit development timelines.

Table 2.2: Non-Functional Requirements

| NFR-ID | Requirement       | Target Metric                                                              |
|--------|-------------------|----------------------------------------------------------------------------|
| NFR-01 | Ingestion Latency | End-to-end update from client agent to dashboard display $\leq 2$ seconds  |
| NFR-02 | OS Compatibility  | Host Agent runs natively on standard Windows and Linux distributions       |
| NFR-03 | Security Controls | Protected cookies, session token rotation, bcrypt hashes, SHA-256 API keys |
| NFR-04 | Maintainability   | Strict type safety and definitions inside all microservices                |
| NFR-05 | Persistence       | Metric payloads survive database container restarts                        |
| NFR-06 | Usability         | Dashboard interface designed with high-density grid layouts                |

## 2.4 Assumptions

It is assumed that the client machine can reach the Ingestion API over HTTP. The local user executing the agent must also hold administrative rights to query system resources via command-line utilities.

## Chapter 3

# System Analysis

### 3.1 Existing Systems Review

Traditional systems like Zabbix, Nagios, and Datadog present established patterns. Zabbix leverages highly optimized client daemons and complex relational architectures, but often relies on continuous database polling rather than native real-time websocket pushes for screen updates. Datadog provides SaaS-based real-time telemetry but is not suitable for disconnected, air-gapped mission ground networks. This project builds a custom, self-hosted streaming pipeline modeled after these systems to bridge the gap.

### 3.2 Phase 1 Analysis

Phase 1 focused on building a schema-agnostic, event ingestion stream. Data received through HTTP or Kafka endpoints was routed to a normalization engine, writing raw entries to PostgreSQL and publishing real-time signals via Redis PubSub.

This successfully demonstrated that decoupled architectures could handle unstructured events. However, log files provide retrospective audit details, while monitoring requires structured, metric-specific data.

### 3.3 Gap Analysis

The ingestion engine lacked operational concepts such as hosts, agents, services, metric definitions, alert rules, and alarm lifecycles. Writing high-frequency time-series datasets directly into a standard relational layout without columnar optimization leads to resource limitations over time.

### 3.4 Proposed Solution

The proposed solution was to evolve the Phase 1 streaming pipeline into a Zabbix-style, infrastructure-oriented monitoring system. Monitored hosts are registered as first-class objects. Relational state mapping is handled in PostgreSQL, while time-series metrics are routed to ClickHouse. Threshold metrics trigger alarm states that are pushed to the user interface via WebSockets.

# Chapter 4

## System Design

### 4.1 High-Level Architecture

The system utilizes a decoupled microservices architecture. Telemetry streams from client machines are collected by the local monitoring agent and forwarded to the ingestion API. These messages are queued in Kafka, analyzed by the metrics processor, stored in PostgreSQL and ClickHouse, and pushed to the Next.js dashboard using Redis PubSub and WebSockets.

### 4.2 Microservices Architecture

The backend consists of five distinct microservices designed to run within separate container boundaries.

Table 4.1: Microservices Architecture

| Service Name      | Network Port | Architectural Role       | Technology Stack                   |
|-------------------|--------------|--------------------------|------------------------------------|
| agent-api         | 4000         | Ingestion Edge           | Fastify, KafkaJS, PostgreSQL       |
| auth-api          | 4002         | REST Control API         | Fastify, JWT, PostgreSQL           |
| metrics-processor | Internal     | Kafka Queue Consumer     | KafkaJS, PostgreSQL, ClickHouse    |
| ws-gateway        | 4001         | WebSocket Broadcaster    | Express, Socket.io, Redis PubSub   |
| monitoring-agent  | Local Client | Host Telemetry Collector | Plain Node.js (Built-in libraries) |

### 4.3 Data Flow Design

The telemetry lifecycle proceeds as follows:

1. The agent executes OS-level metric sweeps and sends an HTTP POST to `/agent/checkin`.
2. The `agent-api` validates the host key and publishes the payload to the Kafka topic `raw-metrics`.
3. The `metrics-processor` consumes the payload, routes transactional variables to PostgreSQL, and streams time-series data to ClickHouse.
4. If a threshold is crossed, the processor opens a problem record in PostgreSQL and publishes a real-time update to Redis.
5. The `ws-gateway` receives the Redis message and pushes the update to the connected dashboard clients.

## 4.4 Database Design

The database layer uses PostgreSQL for relational states and ClickHouse for analytical queries.

### 4.4.1 PostgreSQL Schema

PostgreSQL manages transactional data, user settings, configurations, and alarm logs.

Table 4.2: PostgreSQL Table Description

| Table Name         | Role / Core Contents                                             |
|--------------------|------------------------------------------------------------------|
| tenants            | Organization and tenant division profiles                        |
| users              | Registered dashboard operator records and hashed passwords       |
| sessions           | Session state table for JWT token refresh tracking               |
| api_keys           | SHA-256 hashed keys for automated programmatic lookups           |
| host_groups        | Logical categorization groups for active hosts                   |
| hosts              | Registered target host definitions, labels, and status variables |
| heartbeats         | Timestamp logging for server heartbeat histories                 |
| metric_definitions | Registers metadata for observed variables                        |
| metric_values      | Recent system readings, partitioned monthly                      |
| alert_rules        | Configured warning and critical threshold rules                  |
| problems           | Active and resolved incident alert records                       |
| host_services      | Tracks specific daemon ports (e.g., SSH, HTTP, DB)               |
| audit_logs         | Administrative activity records                                  |

### 4.4.2 ClickHouse Schema

ClickHouse stores historical metric datasets in `platform.metrics` using the MergeTree engine. Materialized views `metrics_hourly` and `metrics_daily` pre-aggregate metrics in the background, optimizing the retrieval of historical charts.

## 4.5 Authentication Architecture

Dashboard users authenticate using secure JWT access tokens stored in `httpOnly` cookies to protect against XSS attacks. Refresh tokens are tracked in the `sessions` table and rotated on renewal. Passwords are encrypted with `bcrypt`, and API keys are stored only as SHA-256 hashes.

## 4.6 Alert Engine Design

Alert rules are loaded into memory and evaluated against incoming metric values. If a metric crosses a critical boundary and no active problem is currently open, a new incident is created. If the threshold is resolved in a later packet, the engine updates the database record to `resolved` and notes the end timestamp.

## 4.7 Monitoring Agent Design

The agent is designed as a single, lightweight JavaScript file running natively without external dependencies. It computes CPU load using `os.cpus()` delta differences, parses Linux disk usage

from `df -k /`, and reads Windows disk metrics using `wmic logicaldisk`. Service statuses are validated using TCP connection attempts via `net.Socket`.

# Chapter 5

## Implementation

### 5.1 Technology Stack

The complete technology selections across the microservices ecosystem are documented below.

Table 5.1: Technology Stack

| Layer                   | Technology Selected          | Version            |
|-------------------------|------------------------------|--------------------|
| Runtime Environment     | Node.js (LTS version)        | 20.x               |
| Implementation Language | TypeScript                   | 5.x                |
| Dashboard Frontend      | Next.js Framework            | 14.x               |
| Styling Library         | Tailwind CSS                 | 3.x                |
| API Gateway Framework   | Fastify                      | 4.x                |
| Message Queue Broker    | Apache Kafka                 | 7.6 (Docker Image) |
| Relational Store        | PostgreSQL Database          | 16.x               |
| OLAP Store              | ClickHouse Analytical Engine | 23.8               |
| In-Memory Store         | Redis                        | 7.x                |
| Real-Time Delivery      | Socket.io Framework          | 4.x                |

### 5.2 Development Environment

Development was performed on Windows 11 using VS Code and PowerShell. Docker Desktop managed PostgreSQL, ClickHouse, Kafka, Zookeeper, Redis, and Mosquitto. Node.js 20 and npm were used for all services. TypeScript services used CommonJS mode to avoid runtime issues with packages that are not compatible with ESM in the selected setup.

### 5.3 Phase 1 Implementation

Phase 1 established the ingestion API and Kafka pipeline. Log events were accepted via HTTP POST, parsed into standard JSON structures, written to PostgreSQL, and published via Redis. A basic dashboard displayed these live log events.

### 5.4 Phase 2 Implementation

Phase 2 adapted the architecture for infrastructure monitoring:



- **Agent API Service:** Implemented endpoints for host registration, heartbeats, and check-ins (`/agent/register`, `/agent/heartbeat`, `/agent/checkin`).
- **Metrics Processor Service:** Implemented a background Kafka consumer to process metric payloads, perform service port checks, write to PostgreSQL and ClickHouse, and evaluate alert rules.
- **Alert Engine:** Implemented state tracking to create and resolve problem records.
- **WebSocket Gateway Redesign:** Redesigned the gateway to broadcast metrics and alarm events to specific host-based socket rooms.
- **Monitoring Agent:** Developed the single-file, cross-platform agent script `agent.js`.
- **Dashboard Implementation:** Built the Next.js interface with summary cards, host grids, service tables, and historical Recharts.

## Chapter 6

# Testing and Validation

### 6.1 Test Strategy

Testing focused on validating the end-to-end flow of metric data from the host agent to the dashboard through all backend services. This was verified using both the real agent and manual HTTP payload tools.

### 6.2 Unit-Level Validation

- **Metric Accuracy:** Verified that the agent's collected values matched the target system's actual resource utilization.
- **Parser Robustness:** Tested the parser engine with sample JSON, CSV, XML, and Syslog payloads.
- **Rule Triggers:** Verified that the alert engine correctly evaluated values above and below configured thresholds.

### 6.3 Integration Testing

The end-to-end integration flow was validated across ten distinct scenarios.

Table 6.1: Integration Testing Scenarios

| Test ID | Scenario             | Expected Outcome                                                   | Result |
|---------|----------------------|--------------------------------------------------------------------|--------|
| IT-01   | User login           | Secure JWT cookie is issued, and access is granted                 | Passed |
| IT-02   | Agent registration   | Valid key returned and host registered in PostgreSQL               | Passed |
| IT-03   | Agent heartbeat      | Receives payload and updates host <code>last_seen</code> timestamp | Passed |
| IT-04   | Check-in ingestion   | Ingests metric payload and publishes to Kafka queue                | Passed |
| IT-05   | Processor consumer   | Consumes from Kafka and inserts metric records into DBs            | Passed |
| IT-06   | Redis PubSub         | Metrics Processor publishes metric events to Redis                 | Passed |
| IT-07   | WebSocket push       | WebSocket Gateway broadcasts the event to the client room          | Passed |
| IT-08   | CPU threshold breach | Alarm triggers, and a problem record is created                    | Passed |
| IT-09   | Alert auto-resolve   | Problem state updates to resolved when metric clears               | Passed |
| IT-10   | Concurrent ingestion | Processes concurrent metrics streams without locking               | Passed |

## **6.4 Performance Observations**

During testing, the average time for an agent metric check-in to process and update the dashboard gauge was under 2 seconds. The system supported multiple concurrent agent check-ins, processing and routing individual payloads based on their respective host keys.

## **6.5 Alert Engine Validation**

Alert triggers were validated by simulating a high CPU load on a monitored host. The Metrics Processor identified the threshold breach, created a problem record, and pushed the alarm to the dashboard. Once the metric returned to its normal range, the engine resolved the problem and noted the end timestamp.

# Chapter 7

## Results and Discussion

### 7.1 Features Delivered

The project delivered a working, Zabbix-style infrastructure monitoring platform. This includes a cross-platform host agent, an ingestion API, a Kafka-based metrics pipeline, a PostgreSQL operational state store, a ClickHouse time-series database, a Redis PubSub message handler, a WebSocket gateway, and a Next.js dashboard.

### 7.2 Performance Results

With heartbeats set to 10 seconds and metrics set to 30 seconds, end-to-end telemetry delivery completed in under 2 seconds. The platform supported concurrent metric streams, storing and displaying updates for individual hosts.

### 7.3 Comparison with Zabbix

The prototype implements the core monitoring concepts found in enterprise platforms like Zabbix.

Table 7.1: Comparison with Zabbix

| Feature Capability     | Enterprise Zabbix Platform         | This Platform Prototype           |
|------------------------|------------------------------------|-----------------------------------|
| Host monitoring agent  | Yes (Compiled C binary)            | Yes (Lightweight Node.js client)  |
| Metrics pipeline       | Direct write to database           | Decoupled with Kafka              |
| 'Storage architecture' | Single Database (PostgreSQL/MySQL) | Dual DB (PostgreSQL + ClickHouse) |
| Dashboard updates      | Refresh polling interval           | Push-based via WebSockets         |
| Active problems board  | Yes (Manual/Auto-resolve)          | Yes (Manual/Auto-resolve)         |
| SNMP poller system     | Yes (Built-in polling engine)      | Planned for future roadmap        |
| Host auto-discovery    | Yes (Subnet IP scan discovery)     | Planned for future roadmap        |

### 7.4 Benefits Achieved

The platform provides a unified view of host health, system metrics, and service availability, reducing the need for manual server audits. The architecture demonstrates how message queues, relational databases, and time-series databases can be integrated to build a scalable monitoring system.

## Chapter 8

# Challenges and Learnings

### 8.1 Technical Challenges

Key technical challenges included resolving package compatibility issues in the TypeScript environment, handling Kafka topic duplication on container restarts, and building cross-platform parsing logic for Windows and Linux within the agent.

### 8.2 Architectural Evolution

The event ingestion pipeline from Phase 1 informed the design of Phase 2. The core components—Kafka, Redis, WebSockets, and authentication—were retained, while the data models were updated to support structured host metrics, alert rules, and problem states.

### 8.3 Engineering Trade-offs

Kafka was integrated to buffer incoming payloads and protect primary databases during agent traffic spikes. ClickHouse was added alongside PostgreSQL to optimize time-series queries, and an agent-based approach was chosen over agentless polling to collect more detailed performance metrics.

### 8.4 Lessons Learned

The project highlighted that monitoring systems require careful design of data collection, storage, and real-time delivery pipelines. It also demonstrated the value of modular architectures, as the ingestion and streaming components developed in Phase 1 were successfully adapted for the monitoring features in Phase 2.

# Chapter 9

## Future Enhancements

### 9.1 Immediate Next Steps

- Implement high-scale load testing to identify bottlenecks in the pipeline.
- Develop dedicated Dockerfiles for each microservice to simplify containerized deployments.
- Build outbound notification channels, such as email, Slack, or webhook integrations, for active alerts.
- Expand the agent's monitoring capabilities to include specific application services like Docker, Redis, and Nginx.

### 9.2 Medium-Term Roadmap

The medium-term roadmap focuses on turning the prototype into a larger operational system.

Table 9.1: Medium-Term Roadmap

| Proposed Milestone     | Priority | Complexity | Strategic Value                                                               |
|------------------------|----------|------------|-------------------------------------------------------------------------------|
| Kubernetes Deployment  | High     | Medium     | Provides horizontal scaling and container orchestration                       |
| SNMP Poller Engine     | High     | High       | Enables agentless monitoring for routers, switches, and other network devices |
| Notification Gateways  | Medium   | Medium     | Delivers real-time critical alerts via SMS or Email                           |
| Host Auto-discovery    | Medium   | High       | Automatically registers hosts by scanning local subnets                       |
| Load and Scale Testing | High     | Medium     | Benchmarks pipeline limits under concurrent host loads                        |

### 9.3 Scalability Opportunities

The ingestion API can be deployed behind a load balancer, the Metrics Processor can be scaled within a Kafka consumer group, and ClickHouse can be clustered to handle higher metric volumes. WebSocket connections can be scaled across multiple gateway instances using a Redis adapter.

### 9.4 Research Opportunities

Future enhancements could explore predictive alerting, anomaly detection, and correlation engines to identify patterns and estimate resource exhaustion before it impacts system availability.

## Chapter 10

# Conclusion

This summer internship project resulted in the design and implementation of a working real-time infrastructure monitoring and observability platform. The system features a cross-platform monitoring agent, an ingestion API, a Kafka-based metrics pipeline, a PostgreSQL operational state store, a ClickHouse time-series database, a Redis PubSub message handler, a WebSocket gateway, and a Next.js dashboard. It implements core monitoring features, including host management, heartbeats, metric trends, alert rules, problems, service checks, and audit logging.

The project was completed in two phases. Phase 1 built a universal event ingestion platform, validating the core streaming and database components. Phase 2 redesigned the system into an infrastructure monitoring platform, adding structured metrics collection, host management, alerting, and an interactive dashboard. This transition helped align the project with real-world infrastructure monitoring requirements.

Validation confirmed that the agent can collect and send system metrics, the pipeline processes and stores payloads, and the dashboard updates in real time. The alert engine successfully opened and resolved problems as values crossed configured thresholds, and end-to-end metric delivery was observed in under 2 seconds. While the system is a prototype and does not yet include advanced enterprise features like SNMP or auto-discovery, it provides a functional, scalable architecture for future development.

## Appendix A

# Installation Guide

1. Install Node.js 20, Git, Docker Desktop, and VS Code.
2. Clone the repository.
3. Copy `.env.example` to `.env` and set the required variables.
4. Launch the database and messaging infrastructure:

```
docker compose -f infra/docker-compose.yml up -d
```

5. Install dependencies in each service directory:

```
cd services/auth-api && npm install
cd ../agent-api && npm install
cd ../metrics-processor && npm install
cd ../ws-gateway && npm install
cd ../../apps/dashboard && npm install
```

6. Start the services in separate terminal instances:

```
npm run dev
```

7. Open `http://localhost:3000` in your browser.



## **Appendix B**

# **Deployment Guide**

For production deployments, each microservice should be built into a separate Docker container. Secrets like passwords, session tokens, and keys should be managed through a secure vault. HTTPS should be enabled for all user and agent connections, and databases should be configured with persistent storage volumes and regular backup policies. Kafka retention and partitioning should be sized to handle the expected volume of metrics.

# Appendix C

## User Manual

- **Logging In:** Access the login screen, enter your credentials, and click submit.
- **System Overview:** The main dashboard shows host health counts and active critical problems.
- **Hosts Inventory:** The Hosts page lists registered hosts, their IP addresses, and their latest status.
- **Detailed Host Metrics:** Click a host to view real-time gauges, service checks, and historical charts.
- **Acknowledge Problems:** The Problems page lists active issues. Operators can click “Acknowledge” to mark an issue as under investigation.
- **Manage Alert Rules:** The Alert Rules page allows administrators to define, update, or toggle warning and critical thresholds.

## Appendix D

# Agent Installation Guide

1. Register the host from the dashboard to generate an `agent_key`.
2. Copy the `agent.js` file to the target machine.
3. Ensure Node.js 20 is installed on the target machine.
4. Run the agent from the terminal, specifying the server address and agent key:

```
node agent.js --server http://localhost:4000 --key YOUR_AGENT_KEY
```

5. Verify the agent is running by checking for heartbeat and metric logs in the terminal.

# Appendix E

## API Reference

### User Authentication

POST /auth/login  
Content-Type: application/json

```
{  
  "email": "admin@platform.local",  
  "password": "Admin@123"  
}
```

### Agent Check-in Payload

POST /agent/checkin  
Content-Type: application/json

```
{  
  "agent_key": "ak_istrac_luc_01",  
  "hostname": "LUC-GW-SRV-01",  
  "metrics": {  
    "cpu_usage": 45.2,  
    "ram_total_mb": 16384,  
    "ram_used_mb": 8192,  
    "disk_total_gb": 512,  
    "disk_used_gb": 256,  
    "uptime_secs": 86400  
  }  
}
```

## Appendix F

# Configuration Reference

Table F.1: Environment Variables

| Variable            | Purpose / Description                                     |
|---------------------|-----------------------------------------------------------|
| PG_URL              | Connection string for the PostgreSQL relational database  |
| REDIS_URL           | Connection string for the Redis cache and PubSub database |
| KAFKA_BROKERS       | List of active bootstrap brokers for the Kafka cluster    |
| CLICKHOUSE_URL      | Endpoint URL for the ClickHouse OLAP database             |
| JWT_SECRET          | Secret key used to sign session tokens                    |
| COOKIE_SECRET       | Secret key used to sign session cookies                   |
| DASHBOARD_ORIGIN    | CORS origin configurations for dashboard access           |
| NEXT_PUBLIC_API_URL | Base API URL for dashboard network requests               |
| NEXT_PUBLIC_WS_URL  | WebSocket gateway URL for real-time dashboard updates     |